

# ML-Inspired Logic Synthesis: Improving Multiplier Circuits

Yukio Miyasaka<sup>1</sup>, Walter Lau Neto<sup>2</sup>, Eleonora Testa<sup>2</sup>, Anika Prasad<sup>2</sup>, Michael Shuster<sup>2</sup>, Reto Zimmermann<sup>2</sup>,  
Patrick Vuillod<sup>2</sup>, Alan Mishchenko<sup>1</sup>, Jown Wawrzynek<sup>1</sup>, Luca Amarù<sup>2</sup>

<sup>1</sup>University of California, Berkeley, Berkeley, California, USA, <sup>2</sup>Synopsys Inc., Sunnyvale, California, USA

**Abstract**—This paper presents an ML-inspired logic synthesis framework to synthesize logic circuits from specification, unlocking logic optimization to reach new, unexplored, solutions in the design space. Based on a given template, the proposed framework creates and trains a model using a novel backpropagation method, don't-care-based backpropagation, which makes it possible to learn non-differentiable attributes of logic circuits efficiently. We ensure the resulting circuit is functionally correct by adding a patch for the cases in which the model is not 100% accurate. To demonstrate its effectiveness, we apply the proposed framework to synthesis of new multiplier circuits. The resulting multipliers are significantly smaller than the best known implementations, with up to 32.8% area reduction. Finally, the generated multipliers were integrated in the core of a state-of-the-art logic synthesis tool, leading to 1.98% area and 0.54% WNS reductions, post place and route, on designs specialized in performing AI tasks.

## I. INTRODUCTION

*Machine Learning* (ML) finds various applications in *Electronic Design Automation* (EDA). These applications range from quality estimation [1], [2], aiming to reduce the turnaround time and speed up the design space exploration, to flow-level optimization [3], [4], where among various possible transformations, the model tries to infer the best sequence of transformations to apply. In this work, we propose a framework that synthesizes a logic circuit from a functional specification using ML techniques, along with a novel backpropagation algorithm. The circuits generated by the proposed framework are structurally different from those derived by the traditional methods based on *Sum-of-Products* (SOP) or *Binary Decision Diagrams* (BDDs). This approach reduces structural bias effects [5] and enables logic optimization to explore previously unreachable solutions on the design space, leading to power, performance, and area improvements.

Because logic circuits comprise logic gates and connections that are not differentiable, ML requires special techniques to synthesize a circuit through training. In [6], the authors proposed a method that makes logic gates differentiable with softmax, while propagating gradients with fuzzy logic [7]. That method was applied to classification problems to improve the accuracy-area tradeoff. An alternative method [8] adopts learnable connections based on gumbel-softmax, which was used to generate circuits from truth tables [9].

We propose a novel framework to synthesize logic circuits using ML techniques. Our framework has three main stages: (i) model generation, (ii) fitting, and (iii) correction/optimization. A template is provided as a starting structure of the model,

which is fed to the fitting stage where the model is trained to realize the target Boolean function. Finally, we apply Boolean techniques to correct the outputs that do not completely satisfy the specification, while optimizing the overall logic. To improve the accuracy and convergence of the model in training, we introduce a novel backpropagation algorithm called *don't-care-based backpropagation*. Instead of relying on fuzzy logic to backpropagate the gradients, we explicitly calculate a desired value at each fanin. This desired value may be *don't care*, which is utilized to help model convergence. The experimental results show that the proposed algorithm can achieve a higher accuracy than traditional backpropagation in circuit model training; our approach produced a functionally correct 4-bit multiplier 92% of the time.

To showcase our proposed framework, we synthesize multiplier circuits that are key components in microprocessors, digital signal processing blocks, and AI accelerators. We focus on *small* multipliers, because narrow bit-width multipliers are common in AI accelerators where low precision is often used to improve efficiency. We achieve up to 32.8% area reduction for 4- to 10-bit multipliers. To further demonstrate the efficiency of the generated multipliers, we integrate them in the state-of-the-art industrial synthesis tool and synthesize AI accelerator designs. Post *Place and Route* (P&R) results show 1.98% area reduction and 0.54% *Worst Negative Slack* (WNS) improvement, on average.

The main contributions of this work are:

- We propose a novel logic synthesis framework inspired by ML techniques that allows deeper exploration of the solution space, discovering new circuit structures.
- We develop a backpropagation algorithm based on don't care that enables circuit model training to achieve higher accuracy and to converge faster than traditional methods.
- We integrate Boolean techniques into the framework to revise the model, guaranteeing functional correctness of the output circuit.
- We derive multiplier circuits, with up to 32.8% area savings, for a range of bit-widths as compared to the best known implementations.

The remainder of this paper is organized as follows: Section II presents relevant background on logic synthesis and ML. Section III explains the proposed framework and don't-care-based backpropagation. Finally, Section IV shows the experimental results, and Section V concludes the paper.

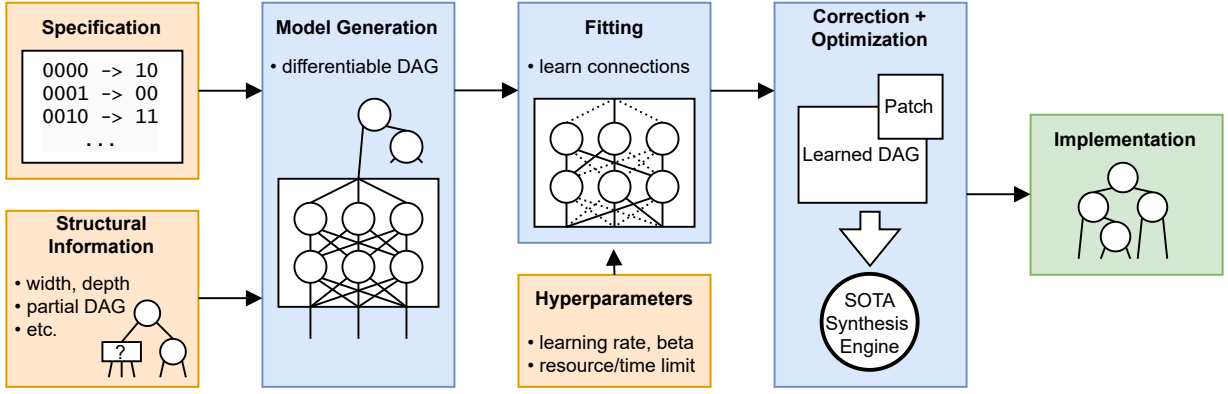


Fig. 1. Overview of the proposed framework.

## II. BACKGROUND

### A. Logic Synthesis

*And-Inverter Graph* (AIG) is a *Directed Acyclic Graph* (DAG), where each node represents a 2-input and-gate, and edges may be inverted [10]. AIGs are commonly used as a technology-independent representation in logic synthesis.

*Observability don't care* in Boolean functions is relevant to this work: given an input pattern, a signal is don't care under observability when the primary outputs evaluate to the same value regardless of its value. Therefore, the local function can be changed without affecting the global function [11].

Modern multi-level logic synthesis algorithms operate on DAG representation of a circuit. Common transformations include rewriting, resubstitution, and refactoring [12]. As these transformations are performed on a DAG, the structure of the input circuit largely affects the optimization results. This dependence is known as structural bias [5]. One successful technique to reduce structural bias during optimization is to interleave don't-care-based methods and mapping to *Look Up Tables* (LUTs) [13], [14].

The concept of *Boolean difference* is commonly used in logic reasoning to indicate functional equivalence. It is defined as  $f \oplus g$  between two Boolean functions  $f(x_0, \dots, x_{n-1})$  and  $g(x_0, \dots, x_{n-1})$ , where  $\oplus$  is an xor-operator [11].

### B. Machine Learning

ML techniques are relevant to this work, especially neural network training. In neural networks, each connection is associated with a weight, which during training gradually updates by the amount of its partial derivative, called a *gradient*. The gradient is given with respect to the error at the network outputs. Because the network typically consists of layers that perform a weighted sum operation and apply a non-linear function, gradients can be derived as a product of a gradient at the local output and a local partial derivative. Hence, gradients are computed in the reverse order of layers, and the process is called *backpropagation*.

In relation with logic circuits, fuzzy logic is often used to make the Boolean operations differentiable. For example, the and-operation ( $\wedge$ ) may be converted into a multiplication ( $\cdot$ ),

where it holds the input-output relation such that  $0 \wedge 0 = 0 \cdot 0 = 0$ ,  $1 \wedge 0 = 1 \cdot 0 = 0$ , and  $1 \wedge 1 = 1 \cdot 1 = 1$ . This approach is straight-forward and easily adoptable using common ML tools, but it should be noted that this relaxation causes a functional discrepancy between the differentiable model and the resulting logic circuit implementation.

Another important technique is softmax. It has been used as a differentiable alternative to argmax, which evaluates to the largest index of a given vector. For a vector  $v$ , softmax can be expressed by the following equation:

$$\text{softmax}(v) = \frac{e^v}{|e^v|}.$$

With fuzzy logic and softmax combined, we can create a differentiable and trainable logic circuit, which is often referred to as a *circuit model*. Circuit neural network [8] was proposed to synthesize a circuit for small Boolean functions using learnable connections. That work used the gumbel-softmax trick that adds random disturbance to the softmax distribution, improving the accuracy over softmax in circuit model training. DiffLogic [6], on the other hand, proposed to make the selection of 2-input node functions differentiable with softmax. This method achieved a good area-accuracy tradeoff in ML tasks such as image classification. There are several follow-up studies in this direction: combining learnable connections [15], using gumbel-softmax [16], and targeting LUT networks [17].

## III. PROPOSED METHOD

### A. Proposed Framework

Figure 1 presents the overview of the proposed framework. The framework takes three inputs: specification, structural information, and hyperparameters. The specification is a functional specification represented as a Boolean function. The structural information is used to create the initial model, and the hyperparameters are used to control the behavior of the fitting stage, as explained below. The framework has three stages: (i) model generation; (ii) fitting, and (iii) correction/optimization. This structure allows the combination

of different approaches for each stage. The output of the framework is an AIG that implements the specification.

The first stage creates a model from a user input. Here various different strategies can be adopted. The model may be composed of layers of nodes that are fully connected by learnable connections. In this case, the user input is the number of nodes per layer and the number of layers in the model. The model may also have skip connections that connect nodes in non-adjacent layers. Alternatively, one can input a template that is a (partial) structure. Then, the model is generated by filling/replacing a part of the structure with layers of nodes connected by learnable connections.

The fitting stage aims to find a configuration for the trainable portions of the model that realizes the specification. We explored the use of backpropagation, where the learnable parameters are gradually adjusted based on the error reversely propagated from the primary outputs. To improve the training efficiency and convergence, we use a novel algorithm, don't-care-based backpropagation—explained in the following subsection. Other methods, such as simulated-annealing, could also be applied, but these methods often result in low accuracy and are computationally expensive.

Finally, the last stage of our framework is correction. This stage broadens the potential of training-based synthesis to the functions that are too complex to learn entirely. After training, the model is converted into an AIG. If the resulting AIG does not fully satisfy the specification, a patch circuit is synthesized and added to the AIG. Different approaches can be selected for this stage. One possibility is to enumerate all the input patterns for which the implementation does not meet the specification, and create a patch that flips the erroneous output for those patterns with Boolean difference. Such enumeration could be exhaustive, where the patch is synthesized from the truth table, or symbolic, where the patch is derived from the BDD. Other possibilities include techniques such as rewiring [18], SAT-based engineering-change-order methods [19], etc. The derivation of patch is a broad research subject, and the relation of error characteristics and patch complexity is an interesting subject for future research.

### B. Don't-Care-Based Backpropagation

As the fitting stage can be treated as an ML problem, we explored several approaches and devised a new backpropagation algorithm, don't-care-based backpropagation. Unlike traditional backpropagation, it explicitly calculates a desired value for each fanin, incorporating an important concept of logic synthesis, *don't care*. It mitigates the problem that the model does not reflect the non-differentiable property of logic circuits, and improves the accuracy of the resulting circuit as well as the training convergence. Algorithm 1 gives an overview of training using don't-care-based backpropagation. The functions and variables are explained in the rest of this subsection.

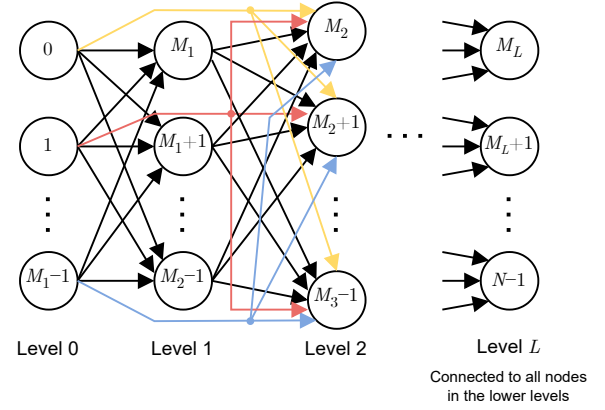


Fig. 2. Structure of the model where each node is connected to all nodes in the lower levels. Edges are colored just for the sake of visibility.

---

#### Algorithm 1 Training with don't-care-based backpropagation

---

**Input:** Input pattern  $u$ ; Output pattern  $v$ ; Parameter  $\{w^i\}$

**Output:** Updated parameter

```

1:  $x := u$ 
2: for  $i = 1$  to  $L$  do
3:    $x, a^i, b^i := \text{Forward}(x, w^i)$ 
4: end for
5:  $f := v$ 
6:  $m := \text{InitializeWeightProducts}(w^L)$ 
7: for  $i = L - 1$  to  $1$  do
8:    $f, m := \text{Backpropagation}(f, m, w^i, a^i)$ 
9: end for
10:  $w' := \text{MinimizeLoss}(\{b^i\}, f)$ 
11: return  $w'$ 

```

---

1) *Model Structure:* Our model is a leveled DAG, as shown in Fig. 2. Each node belongs to a level in the range between 0 and  $L$ . For level 0 ( $L$ ), the number of nodes is given by the number of primary inputs (primary outputs) present in the specification. Thus, these are called *primary input nodes* and *primary output nodes*, respectively. The nodes in the other levels between 1 and  $L - 1$  are referred to as *internal nodes*. While some connections may be removed according to the given structural information, each internal node and primary output node can take at most all nodes that belong to the lower levels as input.

As the output of framework is an AIG, each internal node is converted into a 2-input and-gate after training. The two fanins of the and-gate are selected from all the inputs available at the node based on the learned parameter. Inversions and duplication of fanins is allowed, where the and-gate with duplicated fanins can be translated into a buffer. For the primary output nodes, only one fanin is selected.

Let the nodes be indexed from 0 to  $N - 1$ , in the ascending order of levels, where  $N$  is the total number of nodes in the model.  $M_i$  denotes the total number of nodes in the levels lower than  $i$ . The nodes in level  $i$  are indexed from  $M_i$  to  $M_{i+1} - 1$ . A node with index  $p$  is referred to as node  $p$ .

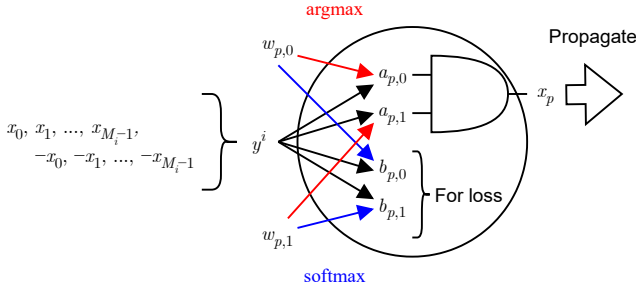


Fig. 3. Forward pass for internal node  $p$  in level  $i$ .

2) *Forward Pass*: We evaluate the current function of the model in forward pass, which yields the same output values as we would see in simulation of the converted logic circuit. The forward pass assigns either  $-1$  (logical zero) or  $1$  to the output of each node. First, the input pattern, for which the model is being evaluated, is assigned to the outputs of the primary input nodes. Then, the other nodes are processed in the order of levels based on their parameters. Each internal node  $p$  has two parameters:  $w_{p,0}$  and  $w_{p,1}$ , each of which corresponds to one fanin as the node is seen as a 2-input and-gate. For each node in level  $i$ , each parameter is a vector of length  $2M_i$ . In Algorithm 1,  $w^i$  represents  $w_{p,0}$  and  $w_{p,1}$  collectively for all nodes in level  $i$ .

Figure 3 summarizes the forward pass for an internal node. Two kinds of values are calculated for each fanin: *argmax fanin value* and *softmax fanin value*. The first one is obtained from the argmax of the parameter vector. One node is selected from the lower levels whose index is equal to the argmax modulo  $M_i$ , and its output value is assigned to the argmax fanin value, while the value is negated when the argmax is larger than or equal to  $M_i$ . Hence, for a node  $p$  in the level  $i$ , the argmax fanin values  $a_{p,0}$  and  $a_{p,1}$  for the first and second fanin, respectively, can be expressed as follows:

$$\begin{aligned} a_{p,0} &= y_{\text{argmax}(w_{p,0})}^i, \\ a_{p,1} &= y_{\text{argmax}(w_{p,1})}^i, \end{aligned}$$

where  $x = (x_0, x_1, \dots, x_{M_i-1})$  is the output values of nodes in the previous levels, and  $y^i = (y_0^i, y_1^i, \dots, y_{2M_i-1}^i)$  is calculated such that  $y_k^i = x_k$  for  $k < M_i$  and  $y_k^i = -x_{k-M_i}$  for  $k \geq M_i$ .

The softmax fanin value is calculated from the softmax of the parameter vector, where we take a weighted sum of the inputs using the softmax as a weight. The softmax fanin values  $b_{p,0}$  and  $b_{p,1}$  for the first and second fanin, respectively, can be expressed as follows:

$$\begin{aligned} b_{p,0} &= y^i \cdot \text{softmax}(w_{p,0}), \\ b_{p,1} &= y^i \cdot \text{softmax}(w_{p,1}). \end{aligned}$$

The output of internal node is calculated using the argmax fanin values. Note that this operation does not need to be differentiable, because the proposed backpropagation algorithm

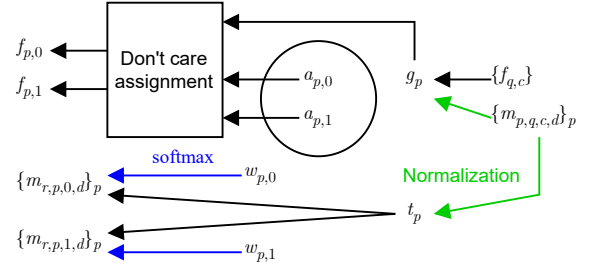


Fig. 4. Don't-care-based backpropagation for internal node  $p$ .  $\{m_{p,q,c,d}\}_p$  and  $\{m_{r,p,c,d}\}_p$  are the matrices bounded by  $p$ .

does not propagate gradients through it. The output of node  $p$ ,  $x_p$ , is expressed as follows:

$$x_p = \begin{cases} 1, & \text{if } a_{p,0} = 1 \wedge a_{p,1} = 1 \\ -1, & \text{otherwise} \end{cases}.$$

The softmax fanin values are not used to calculate the output of node, so they do not propagate to the nodes in the upper levels. On the other hand, they are used in the loss on which the parameters are updated, as explained later.

Each primary output node has only one parameter ( $w_{p,0}$  for node  $p$ ), which is a vector of length  $2M_L$ , since each primary output of an AIG has only one driver. In the same way as internal nodes, the argmax fanin value and the softmax fanin value are calculated for each primary output node. The argmax fanin value of the node is directly assigned to the output of primary output node, which is then used as an output of the model.

In Algorithm 1,  $a^i$  and  $b^i$  represent the argmax fanin values and softmax fanin values, respectively, in level  $i$  collectively. The output values of the nodes are concatenated to the input  $x$  and propagate to the next level.

3) *Don't-Care-Based Backpropagation*: The proposed don't-care-based backpropagation is performed starting from level  $L$  down to level  $1$ , in a descending order of levels, as in traditional backpropagation. It calculates a desired value, denoted by  $f_{p,c}$ , for each fanin and for each node, where  $p$  is the node index and  $c$  is the fanin position:  $0$  for the first fanin and  $1$  for the second fanin. First, the desired values of the fanins in level  $L$  are set to the values of the corresponding primary outputs in the specification. Then, the internal nodes are processed as shown in Fig. 4.

To explain the process for the internal nodes, we introduce two concepts, *potential fanout* and *weight product*. A potential fanout is a fanin of the node in the upper levels, which could become a fanout in the AIG. Each potential fanout can be represented as a tuple  $(q, c, d)$ , where  $q$  is the fanout node's index,  $c$  is the fanin position, and  $d$  is the inverter presence. In our model, at most all nodes in the upper levels are potential fanouts. In that case, a node in level  $i$  has potential fanouts whose indices range from  $M_{i+1}$  to  $N - 1$ .

The desired value is calculated by taking a weighted sum of the desired values of the potential fanouts, where the weight here is not a learnable parameter but the value that should

represent how likely each potential fanout becomes a real fanout. A concept of weight product is introduced to this end. A weight product  $m_{p,q,c,d}$  is calculated for each node  $p$  and each potential fanout  $(q, c, d)$  during backpropagation. Initially, for each pair of an internal node  $p$  and a primary output node  $q$ , the weight product  $m_{p,q,0,0}$  is assigned to the  $p$ -th element, and  $m_{p,q,0,1}$  is assigned to the  $(p + M_L)$ -th element of the softmax of the parameter of the primary output node. Let the  $j$ -th element ( $0 \leq j < 2M_L$ ) of  $\text{softmax}(w_{q,0})$  be  $s_{q,0,j}$ . Then, this assignment can be expressed as follows:

$$m_{p,q,0,0} = s_{q,0,p}, \quad m_{p,q,0,1} = s_{q,0,p+M_L}.$$

The don't-care-based backpropagation is performed for each internal node  $p$  in level  $i$  as follows:

- 1) The weight products are normalized as follows:

$$m'_{p,q,c,d} = \frac{m_{p,q,c,d}}{t_p},$$

where  $t_p = \sum_{q,c,d} m_{p,q,c,d}$ .

- 2) A weighted sum  $g_p$  of the desired values of the potential fanouts is calculated as follows:

$$g_p = \sum_{q,c,d} m'_{p,q,c,d} \cdot (f_{q,c} \dot{\vee} d),$$

where  $f_{q,c}$  is the desired value at  $c$ -th fanin of node  $q$ , and  $\dot{\vee}$  is a custom xor-operator, which flips the sign of  $f_{q,c}$  when  $d = 1$ .

- 3) The desired values  $f_{p,0}$  and  $f_{p,1}$  are calculated as follows:

$$f_{p,0} = \begin{cases} 0, & \text{if } (g_p < 0 \vee a_{p,0} = 1) \wedge a_{p,1} = -1 \\ g_p, & \text{otherwise} \end{cases},$$

$$f_{p,1} = \begin{cases} 0, & \text{if } (g_p < 0 \vee a_{p,1} = 1) \wedge a_{p,0} = -1 \\ g_p, & \text{otherwise} \end{cases}.$$

- 4) For each node  $r$  in the lower levels, weight products  $m_{r,p,c,d}$  are calculated for all  $c, d \in \{0, 1\}$  as follows:

$$m_{r,p,0,0} = t_p \cdot s_{p,0,r}, \quad m_{r,p,0,1} = t_p \cdot s_{p,0,r+M_i},$$

$$m_{r,p,1,0} = t_p \cdot s_{p,1,r}, \quad m_{r,p,1,1} = t_p \cdot s_{p,1,r+M_i},$$

where  $s_{p,0,j}$  denotes the  $j$ -th element of  $\text{softmax}(w_{p,0})$ , and  $s_{p,1,j}$  denotes that of  $\text{softmax}(w_{p,1})$  for  $0 \leq j < 2M_i$ .

Intuitively, steps 1 and 2 backpropagate the desired values from the potential fanouts, step 3 augments these desired values with don't care, based on the current values of the fanins, and step 4 prepares the weight products for backpropagation to the lower levels. Don't care (0) is assigned when the other fanin takes -1 (logical zero) since the nodes correspond to and-gates. However, if the fanin itself takes -1 and the desired value is 1 (or positive, more precisely), the don't care is not assigned. This is because both fanins need to change to 1 to turn the output to 1.

As shown in Algorithm 1, backpropagation takes the parameter  $w^i$  and argmax fanin values  $a^i$  of the current level  $i$  as

input, and calculates the desired value  $f$  and weight product  $m$ , which are appended to those from the upper levels. Then, the desired values are used to calculate the loss on which training is performed as a minimization problem.

The objective of training is to get each softmax fanin value,  $\{b^i\}$ , closer to the *sign* of the calculated desired value. The sign of  $f$  is defined as a function that takes 1 when  $f > 0$ , -1 when  $f < 0$ , and 0 when  $f = 0$ . Then, training is formulated as a minimization problem over a sum of the losses calculated at each node  $p$  as follows:

$$|f_{p,0}| \cdot \ell(b_{p,0}, \text{sign}(f_{p,0})) + |f_{p,1}| \cdot \ell(b_{p,1}, \text{sign}(f_{p,1})),$$

where  $\ell$  is an arbitrary loss function of choice. This formula controls the magnitude using the desired value in the coefficients, while the softmax fanin values are oriented towards either -1 or 1, based solely on its sign for the fanins that are not don't care. Although the softmax fanin values were not directly used to calculate the outputs of nodes, the updated parameters affect the outputs of nodes via the argmax fanin values as in straight-through estimators [20].

## IV. EXPERIMENTAL RESULTS

This section presents the results of three experiments, each with a different purpose. First, we compare the learning convergence of our new backpropagation technique and standard backpropagation techniques, applied to circuit model training. Next, we present the stand-alone size of multipliers synthesized using the proposed framework. Finally, we report a summary of the post P&R results on AI accelerator designs with and without using the generated multipliers.

### A. Learning Convergence

We compare the proposed training method, don't-care-based backpropagation, with the standard approaches used to make the connections differentiable: softmax and gumbel-softmax. We used 4-bit unsigned multiplication as a specification in this experiment. For all the three approaches, we input the same model structure, which contains 20 levels of internal nodes with 20 nodes per level. We used PyTorch [21] to implement our program. We used mean square error as a loss function and Adam [22] as an optimizer. For baseline approaches, we optimized Adam's hyperparameters (learning rate  $lr$  and beta  $\beta$ ) for one day [23] and used the average hyperparameters of the best five runs:  $(lr, \beta_1, \beta_2) = (1.3, 0.02, 0.4)$  for softmax and  $(0.4, 0.4, 0.8)$  for gumbel-softmax, while it was set to  $(0.1, 0, 0.9)$  for the proposed method.

We repeated the experiment 100 times independently varying the random seed. The curves of average accuracy are shown in Fig. 5 along with the range (min, max). The accuracy was evaluated on the function of the converted logic circuit. The result indicates that the proposed method can effectively learn the function as a logic circuit. Notably, 92 runs achieved 100% accuracy (exact implementation) by the proposed method, while none of the runs did when using our implementation with softmax and gumbel-softmax.

TABLE I  
MULTIPLIER SIZE

| Bit-width | AIG node count |     |            |               | Mapped area ( $\mu m^2$ ) |        |               |               |
|-----------|----------------|-----|------------|---------------|---------------------------|--------|---------------|---------------|
|           | SOTA           | ABC | Proposed   | Gain (%)      | SOTA                      | ABC    | Proposed      | Gain (%)      |
| 4x4       | 106            | 84  | <b>69</b>  | <b>-17.86</b> | 59.09                     | 59.27  | <b>39.69</b>  | <b>-32.83</b> |
| 5x5       | 177            | 145 | <b>126</b> | <b>-13.10</b> | 99.67                     | 99.84  | <b>81.14</b>  | <b>-18.59</b> |
| 6x6       | 290            | 222 | <b>201</b> | <b>-9.46</b>  | 149.59                    | 150.65 | <b>131.59</b> | <b>-12.03</b> |
| 7x7       | 423            | 315 | <b>300</b> | <b>-4.76</b>  | 209.39                    | 211.68 | <b>185.57</b> | <b>-11.38</b> |
| 8x8       | 566            | 424 | <b>396</b> | <b>-6.60</b>  | 279.77                    | 283.83 | <b>249.43</b> | <b>-10.84</b> |
| 9x9       | 738            | 549 | <b>528</b> | <b>-3.83</b>  | 359.86                    | 366.74 | <b>329.52</b> | <b>-8.43</b>  |
| 10x10     | 930            | 690 | <b>666</b> | <b>-3.48</b>  | 450.88                    | 459.87 | <b>430.06</b> | <b>-4.62</b>  |

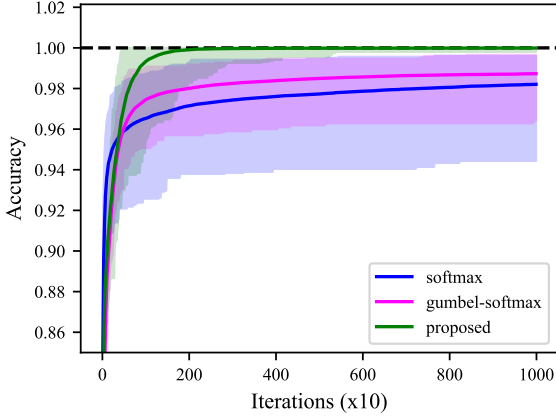


Fig. 5. Learning curves: line (average) and fill (min, max).

### B. Multiplier Synthesis

We applied the proposed framework to multiplier synthesis using an advanced commercial technology library. The synthesis was performed iteratively: a model was constructed by replacing a part of the previous circuit with fully connected layers including all skip connections, training was performed using our don't-care-based backpropagation for at most an hour, and the resulting circuit was corrected with BDDs if necessary and optimized with a standard AIG optimization flow similar to [13], [14]. We iterated the process for a few weeks for large multipliers, which may be justified by their high importance. As a baseline, we collected area-oriented multipliers from the *State-Of-The-Art* (SOTA) industrial tool and also an opensource tool, ABC [24], with a command `%blast`. We measured the number of AIG nodes and post technology-mapping area. Since we performed synthesis iteratively, we recorded the best multiplier for each metric. Note that these multipliers are unsigned.

Table I shows the metrics of the baseline multipliers and the multipliers synthesized by the proposed framework. It should be noted that the SOTA multipliers are designed targeting mapped area. Hence, even though the SOTA multipliers present larger AIG counts, indeed their mapped area is always smaller than the ABC counterparts. The gain of synthesized multiplier is shown w.r.t. the smaller baseline: ABC for AIG node count, and SOTA for mapped area. In both metrics, the proposed method achieved substantial reduction: from 3.4% to 17.8% for AIG node count, and from 4.6% to 32.8% for

TABLE II  
POST P&R AVERAGE RESULTS

| Flow     | Multiplier % | Area          | WNS           | Runtime |
|----------|--------------|---------------|---------------|---------|
| Baseline | 10.73        | 1             | 1             | 1       |
| Proposed | 10.73        | <b>-1.98%</b> | <b>-0.54%</b> | 1       |

mapped area. This reduction comes from the large don't care space present on multipliers. We note that the gain diminishes as the multiplier size increases. On the one hand, this is because scaling our method for larger designs is challenging. On the other hand, the compressor cells were not used in the mapped netlist of our multipliers, which becomes relevant for large multipliers.

### C. End-to-End Results

We present our experimental results on three AI-intensive ASIC benchmarks, obtained using a commercial EDA flow empowered with our new multipliers. The selected blocks are naturally multiplier intensive, and have, on average, 10.73% of the combinational area comprised by multipliers. The generated multipliers are embedded in the core of the synthesis flow, and are made available to be selected by the tool based on the design constraints. To show the efficacy of our method, we thus present the average results w.r.t. a baseline flow where our multipliers are not available. The results, post P&R, are summarized in Table II. All benchmarks are verified to be equivalent with an industrial formal equivalence checking flow. Our complete design flow, achieved an average improvement of combinational area by 1.98% and WNS by 0.54%. Runtime is slightly improved due to the area reduction but remains within noise range.

## V. CONCLUSION

In this paper, we proposed a logic synthesis framework integrating ML techniques. To train a circuit model, we developed a novel backpropagation algorithm, which showed higher accuracy and better convergence than existing methods. For the cases where the trained model does not fully satisfy the specification, the framework uses Boolean methods to correct its function. The proposed framework generated multipliers that present up to 32.8% area reduction compared to best known implementations. Finally, embedded in a synthesis tool, the generated multipliers led to an average area reduction of 1.98% and WNS reduction of 0.54% for three relevant AI accelerator designs.

## REFERENCES

- [1] S. Dai, Y. Zhou, H. Zhang, E. Ustun, E. F. Young, and Z. Zhang, “Fast and accurate estimation of quality of results in high-level synthesis with machine learning,” in *Proceedings of FCCM*, 2018, pp. 129–132.
- [2] W. L. Neto, M. T. Moreira, L. Amaru, C. Yu, and P.-E. Gaillardon, “Read your circuit: Leveraging word embedding to guide logic optimization,” in *Proceedings of ASPDAC*, 2021, p. 530–535.
- [3] K. Zhu, M. Liu, H. Chen, Z. Zhao, and D. Z. Pan, “Exploring logic optimizations with reinforcement learning and graph convolutional network,” in *Proceedings of MLCAD*, 2020, pp. 145–150.
- [4] W. L. Neto, Y. Li, P.-E. Gaillardon, and C. Yu, “FlowTune: End-to-end automatic logic optimization exploration via domain-specific multiarmed bandit,” *IEEE TCAD*, vol. 42, no. 6, pp. 1912–1925, 2023.
- [5] S. Chatterjee, A. Mishchenko, R. Brayton, X. Wang, and T. Kam, “Reducing structural bias in technology mapping,” in *Proceedings of ICCAD*, 2005, pp. 519–526.
- [6] F. Petersen, C. Borgelt, H. Kuehne, and O. Deussen, “Deep differentiable logic gate networks,” in *Proceedings of NeurIPS*, vol. 35, 2022, pp. 2006–2018.
- [7] G. J. Klir and B. Yuan, *Fuzzy Sets and Fuzzy Logic: Theory and Applications*. USA: Prentice-Hall, Inc., 1994.
- [8] A. Hillier and N. Vu, “Learning to design efficient logic circuits,” 2023. [Online]. Available: [https://github.com/alanminko/iwls2023-ls-contest/blob/main/presentations/IWLS2023\\_Contest\\_Presentation\\_Google.mp4](https://github.com/alanminko/iwls2023-ls-contest/blob/main/presentations/IWLS2023_Contest_Presentation_Google.mp4)
- [9] IWLS, “Problems and results of IWLS 2023 programming contest,” 2023. [Online]. Available: <https://github.com/alanminko/iwls2023-ls-contest>
- [10] A. Biere, “The AIGER And-Inverter Graph (AIG) format version 20071012,” Institute for Formal Models and Verification, Johannes Kepler University, Altenbergerstr. 69, 4040 Linz, Austria, Tech. Rep. 07/1, 2007.
- [11] G. D. Micheli, *Synthesis and Optimization of Digital Circuits*, 1st ed. McGraw-Hill Higher Education, 1994.
- [12] A. Mishchenko, S. Chatterjee, and R. Brayton, “DAG-aware AIG rewriting: A fresh look at combinational logic synthesis,” in *Proceedings of DAC*, 2006, p. 532–535.
- [13] L. Amarú, V. Possani, E. Testa, F. Marranghello, C. Casares, J. Luo, P. Vuillod, A. Mishchenko, and G. De Micheli, “LUT-based optimization for ASIC design flow,” in *Proceedings of DAC*, 2021, pp. 871–876.
- [14] W. L. Neto, L. Amarú, V. Possani, P. Vuillod, J. Luo, A. Mishchenko, and P.-E. Gaillardon, “Improving LUT-based optimization for ASICs,” in *Proceedings of DAC*, 2022, p. 421–426.
- [15] C. Yue and N. K. Jha, “Learning interpretable differentiable logic networks,” *IEEE TCASAI*, pp. 1–14, 2024.
- [16] Y. Kim, “Deep stochastic logic gate networks,” *IEEE Access*, vol. 11, pp. 122 488–122 501, 2023.
- [17] A. T. L. Bacellar, Z. Susskind, M. Breternitz Jr, E. John, L. K. John, P. M. V. Lima, and F. M. França, “Differentiable weightless neural networks,” in *Proceedings of ICML*, vol. 235, 2024, pp. 2277–2295.
- [18] T.-K. Lam, W.-C. Tang, X. Wei, Y. Diao, and D. Y.-L. Wu, *Delete-First Rewiring Techniques*, 2016, pp. 67–132.
- [19] A. Q. Dao, N.-Z. Lee, L.-C. Chen, M. P.-H. Lin, J.-H. R. Jiang, A. Mishchenko, and R. Brayton, “Efficient computation of ECO patch functions,” in *Proceedings of DAC*, 2018.
- [20] E. Jang, S. Gu, and B. Poole, “Categorical reparameterization with gumbel-softmax,” in *Proceedings of ICLR*, 2017.
- [21] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. Luk, B. Maher, Y. Pan, C. Puhersch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, M. Suo, P. Tillet, E. Wang, X. Wang, W. Wen, S. Zhang, X. Zhao, K. Zhou, R. Zou, A. Mathews, G. Chanan, P. Wu, and S. Chintala, “PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation,” in *Proceedings of ASPLOS*, 2024.
- [22] D. P. Kingma and J. L. Ba, “Adam: A method for stochastic optimization,” in *Proceedings of ICLR*, 2015.
- [23] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of ICKDDM*, 2019.
- [24] Berkeley Logic Synthesis and Verification Group, “ABC: A system for sequential synthesis and verification.” [Online]. Available: <http://www.eecs.berkeley.edu/~alanmi/abc/>